

DQ Compass

Points clés à dire dans la présentation - trame par partie

MBA-ESG / SG GSC - Datathon 2026 | Groupe 1 | 15 min + 5 min Q&R

Genere le 2026-09-08

Comment lire ce document. Pour chaque partie : les idées à faire passer (marquées d'un triangle vert), et ce qu'il faut éviter (en rouge italique). La section 10 (choix de l'outil) se dit à l'oral juste avant la demo. Sections 14 (Q&R) et 15 (budget temps) en préparation.

Regles d'or

- Placer tot les mots-clés : **tracable, reproductible, auditable, preuve, BCBS 239, plug-and-play, frugal, deterministe.**
- Toujours relier une fonctionnalité à une exigence du brief ou à un axe d'évaluation.
- Annoncer clairement le périmètre MVP - ne rien promettre qui n'est pas fait.
- × *Lire les slides mot à mot ; se perdre dans le code ; dépasser le temps.*
- × *Presenter les 6 briques au même niveau de détail - l'Audit est le point fort.*

0 - Ouverture (30 s)

- Qui : Groupe 1. Use case : **une couche de controle Data Quality plug-and-play pour les EUC.**
- L'accroche : *les EUC portent des processus critiques mais échappent à la gouvernance du SI ; DQ Compass se branche sans écrire de code et produit la preuve par défaut.*
- Le plan : problématique, solution brique par brique, cas d'usage complet, conformité, perspectives - et une demo.

1 - La problématique (1 min 30)

- EUC = End-User Computing : Excel, Access, scripts qui font tourner de vrais processus métier, **hors du SI et de ses controles.**
 - Trois douleurs : (1) hors gouvernance ; (2) chaque équipe refait ses controles à la main, sans trace exploitable ; (3) **BCBS 239** exige des controles traces, reproductibles et auditables - un classeur Excel ne sait pas le démontrer.
 - La cible du brief : le cycle **Définir, Exécuter, Restituer, Prouver**, sur 6 dimensions (complétude, validité, unicité, cohérence, fraîcheur, rapprochement), **frugal** (IA optionnelle), **déterministe, réutilisable.**
- × Entrer dans le détail technique ici - c'est le cadrage du besoin.

2 - La solution, vue d'ensemble (1 min)

- Le principe qui porte tout : **les règles sont des données, le moteur est générique** -- on configure, on ne code pas.
- Quatre briques : **Catalogue** (définit), **Moteur** (exécute), **Reporting** (restitue), **Audit** (prouve), alimentées par des connecteurs.
- Conséquence concrète : **un nouvel EUC = 2 fichiers de configuration, aucun développement.**

3 - Le Catalogue de controles (1 min)

- Une règle = **une ligne de fichier** ; une application web pour les créer, les modifier, les versionner.
- Un **meta-catalogue (~30 vérifications)** bloque toute règle invalide **avant** enregistrement (colonne inexistante, seuil incohérent, regex qui ne compile pas).
- 14 attributs (Annexe A.2), 6 types de contrôle, sévérité, propriétaire, seuil de tolérance, action corrective.
- La définition en **SQL lisible par un auditeur** est générée automatiquement ; chaque changement est journalisé (journal chaîne par hash).

4 - Le Moteur d'exécution (1 min)

- Une **fonction générique par dimension** : elle ne connaît que des noms de colonnes -- elle tourne sur n'importe quel dataset, sans jamais le modifier.
- **Déterministe et reproductible** : même entrée, même résultat.
- **Résilient** : une source manquante devient un statut ERROR, l'exécution continue - le moteur ne plante jamais.
- **Seuils de tolérance** : ex. $\leq 1\%$ de lignes en anomalie = conforme.

5 - Sources de données, plug and play (45 s)

- **Abstraction connecteur** : CSV et Excel opérationnels ; JSON / SQL / SharePoint prêts en sélecteurs (phase 2) - périmètre MVP assumé.
- Ajouter une source = **une ligne de configuration.**
- **Politique de suppression maîtrisée** : retrait réversible (cascade sur les règles), purge définitive, renommage (réaffecte les règles dépendantes).

6 - Le Reporting (1 min)

- **Deux publics** via un filtre d'audience : le métier (risk / data owner) et le data engineer.

- **Score DQ composite** pondere par severite (un chiffre a suivre), tendance, sante par dataset, matrice de couverture, incidents ouverts + remediation, exceptions par proprietaire / dimension, fiabilite des regles.
- **Selecteur d'execution** : on peut recalculer le tableau de bord sur n'importe quelle execution passee.
- **7 exports CSV / JSON** : format d'echange pret pour Power BI - pas de re-developpement cote BI.
- × *Sur-vendre le score 87/100 : c'est une convention ponderee ; ce qui compte c'est la tendance et la comparaison entre datasets.*

7 - L'Audit / l'auditabilite (1 min 30 - LE POINT FORT)

- **Evidence Pack par execution** : les 10 composants de l'Annexe B.2 - donnees figees (+ hash de contenu), configuration figee, parametres resolus, metriques, dataset d'exceptions, journaux, trace systeme, sign-off.
- **Registre des executions chaine par hash** : chaque run signe le precedent -- toute alteration d'un run passe devient **visible** (inviolable).
- **Verification independante « Verify »** : rejoue la configuration figee sur les donnees figees et retrouve le meme resultat (Annexe B.3).
- **Sign-off humain + 2 journaux de changements chaines** (catalogue + sources) : qui a change quoi, quand.
- La phrase a marteler : **on ne fait pas que detecter les anomalies, on prouve le controle.**

8 - Le Mapping superviseur (45 s)

- **Genere depuis le catalogue** - jamais desynchronise.
- Pour chaque controle : **l'exigence BCBS 239 couverte, la preuve produite, le hash du run qui l'atteste** (Annexe C).
- C'est le pont entre le technique et le langage superviseur.

9 - Cas d'usage (1 min 30)

- Donnee **reelle** : fichier BIS « OTC derivatives turnover », ~78 000 lignes, 73 colonnes - pas un jeu de donnees jouet.
- **14 regles : 2 par dimension**, une concue pour passer, une pour echouer -- une seule execution produit un reporting complet et tous les cas.
- Exemples parlants : code pays « 5Z » absent du referentiel = **8 997 lignes** ; **5 flux** pays datant de plus de 7 jours ; grand livre stresse = **3 ecarts + 1 deal manquant**.
- Resultat : scorecard + Evidence Pack + mapping superviseur en un seul passage, le tout **verifie 14 / 14**.

10 - Choix de l'outil et de la stack (1 min 30) -- JUSTE AVANT LA DEMO

Cette section justifie les decisions techniques. Elle repond a la contrainte « frugal by design » du brief et coupe court aux questions du jury.

Choix	Pourquoi (a dire)
Python + pandas ; 0 IA, 0 base de donnees, 0 cloud	Contrainte de frugalite du brief. Tourne sur un simple poste. Langage des equipes data, lisible.
Les regles en CSV (pas une appli a base de donnees)	Versionnable dans Git, comparable (diff), lisible sans l'outil, zero infrastructure. Le meta-catalogue garantit la validite ; l'ecriture est atomique.
Une petite appli web (Flask)	Une interface d'autorite pour creer / valider les regles et signer les runs, sans framework lourd - lancable en une commande.
Preuve en fichiers JSON / CSV (pas un SIEM)	Transparents, auditables a l'oeil, portables. En production on les pousse sur du stockage WORM (immuable) - l'interface est deja la.
Hachage SHA-256 + chainage des runs	Standard, deterministe, sans tiers de confiance. Detecte l'alteration ; la signature cryptographique est l'etape production.
Pas d'IA dans le scoring	Un score DQ doit etre deterministe et explicable. L'IA reste une piste optionnelle (suggestion de regles par profilage des donnees).
Reproductibilite	Prouvee par 42 tests automatises et par la commande Verify - pas une promesse.

■ Transition : « voila pour les choix techniques - place a la demonstration. »

11 - Passage a la demo (30 s d'annonce + 4-5 min)

Annoncer la sequence, puis derouler. Garder une regle « en echec » sous la main pour le drill-down.

#	Ce qu'on montre	Ce qu'on dit en le montrant
1	Le Catalogue - creer / valider une regle	« une regle = un formulaire ; le meta-catalogue refuse une regle invalide. »
2	Lancer une execution (Run engine now)	« le moteur applique tout le catalogue au dataset. »
3	La page Reporting	score, tendance, incidents ouverts + remediation ; ouvrir une regle de Validite en echec -- les lignes fautives + « comment corriger ».
4	La page Runs -- un Evidence Pack -- bouton Verify	« tout est fige et rejouable ; Verify retrouve le meme resultat. »
5	La page Mapping	« chaque controle relie a une exigence BCBS 239 et au hash du run. »

× Plan B si la demo plante : les captures dans les slides + le PDF de conformite. Le dire calmement et continuer.

12 - Conformite et axes d'evaluation (1 min)

■ Cocher explicitement : **Annexe A.2** (14 attributs), **B.2** (10 composants de preuve), **B.3** (verification independante), **Annexe C** (mapping), **6/6 dimensions**, **frugal / sans IA**, **deterministe**, **42 tests**.

■ Relier aux 4 axes du jury : **qualite technique** (moteur generique, reproductible, resilient), **maturite DQ** (6 dimensions, reutilisable), **gouvernance et auditabilite** (Evidence Pack, chaine de hash, Verify, sign-off), **valeur metier** (risque reduit, incidents actionnables, deploiement 2 fichiers, exports BI).

13 - Perspectives (1 min)

■ **A l'horizontale** : un EUC de plus = 2 fichiers ; catalogue central versionne, tire par chaque EUC ; la CLI s'insere dans la CI (contrôle bloquant).

■ **A la verticale** : les 6 fonctions ont un contrat fixe -- on remplace pandas par DuckDB ou Spark SQL **sans toucher** au catalogue, a la preuve, ni a l'appli.

■ **MVP honnete** : 3 connecteurs a finir, execution en memoire, preuve locale, appli mono-utilisateur -- phase suivante documentee, a faible risque.

■ Cloture : « d'un EUC que personne ne peut auditer, a une couche de controle qu'un regulateur peut lire. »

14 - Q&R : questions probables et reponses courtes

Question	Reponse en une phrase
Pourquoi pas d'IA ?	Un score DQ doit etre deterministe et explicable. L'IA est une piste optionnelle (suggestion de regles par profilage).
Ca tient a l'echelle ?	Echelle EUC aujourd'hui ; grace au contrat de fonction fixe, on passe a DuckDB / Spark sans reecrire le catalogue ni la preuve.
Des regles en CSV, c'est robuste ?	Meta-catalogue (~30 verifications) + ecriture atomique + journal chaine par hash + versionnement Git.
Qu'est-ce qui empeche de trafiquer une preuve ?	La chaine de hash rend toute alteration visible ; en production, stockage WORM immuable.
Difference avec Great Expectations / un outil du marche ?	Ici l'accent est sur la preuve et l'auditabilite de bout en bout + le mapping superviseur, pas seulement la detection - et zero infrastructure.
Combien de temps pour brancher un nouvel EUC ?	Le temps d'ecrire 2 fichiers de configuration (source + regles).
Le score 87/100, ca veut dire quoi ?	Convention ponderee par severite : une regle High pese 5x une Low. On suit la tendance, pas la valeur absolue.
Et si une source est indisponible le jour J ?	La regle passe en ERROR, l'execution continue, et c'est trace dans l'Evidence Pack.

15 - Budget temps (15 min + 5 min Q&R)

Partie	Duree	Diapos
0 Ouverture	0:30	1
1 Problematique	1:30	2
2 Solution - vue d'ensemble	1:00	3
3-8 Les 6 briques (Catalogue, Moteur, Sources, Reporting, Audit, Mapping)	5:00	4-9
9 Cas d'usage	1:30	10-11
10 Choix de l'outil / stack	1:30	oral, avant demo
11 Demo	4:00 - 5:00	app live
12 Conformite et axes d'evaluation	1:00	12-13
13 Perspectives + cloture	1:00	14-15
Total	~15:00	

Si le temps deborde : compresser 3-8 (une phrase par brique sauf Audit) et raccourcir la demo aux etapes 3 et 4 (Reporting + Verify).